

Skillomate production API routing fix

Previous repository state compared with the production-ready frontend build generated on 21 September 2026.

Outcome	Production requests now resolve through https://api.skillomate.in ; local development continues to use the Vite /api proxy.
Build	PASS - Vite production build completed (129 modules transformed).
Tests	PASS - 24 test files and 157 tests passed, including seven production routing assertions.
CORS	Updated - production allowlist always includes both apex and www Skillomate origins.

1. Root cause

The shared frontend helper returned relative **/api/...** paths. In production, the browser therefore sent those requests to the frontend origin and CloudFront distribution at **www.skillomate.in**, where they hung instead of reaching the dedicated ALB-backed API hostname. Several direct fetch, admin, image, certificate, payment, onboarding, and playback paths also bypassed the shared helper.

2. Before and after

Area	Previous version	New build
Production base	No VITE_API_BASE_URL	VITE_API_BASE_URL=https://api.skillomate.in
/api/courses	Same-origin /api/courses	https://api.skillomate.in/api/courses
Path handling	Only /api-prefixed input	Supports /api/x, api/x, and /x without duplicate /api
Direct fetches	Several bypassed the helper	Delegated through the centralized URL builder
Local development	Vite proxy to configurable target	Preserved; blank base keeps /api relative
Production CORS	Dependent only on runtime env values	Exact apex and www origins always included in production
Authentication	Bearer Authorization headers	Unchanged and covered by a routing test

3. Notable code and configuration changes

Central URL builder

edunex-f/`js/edunex-api.js` now reads a build-injected meta value, validates and normalizes the origin, creates one canonical `/api` path, preserves absolute external URLs, and routes backend image proxy URLs through the same mechanism.

Build-time environment

edunex-f/`vite.config.js` loads `VITE_API_BASE_URL` and injects it into every generated application shell.

edunex-f/`env.production` supplies the public production hostname. **.env.example** documents a blank local base and retains `VITE_API_PROXY_TARGET=http://127.0.0.1:3000`.

Direct-call consolidation

A small module adapter in **src/**`lib/apiUrl.js` delegates module code to `window.EduNex.apiUrl`. Payment, login, signup, password recovery, deletion requests, reports, onboarding, course retries, admin requests, certificates, thumbnails, image proxy requests, and relative playback grants now use it.

Backend CORS

edunex-b/`server.js` now merges configured origins with exact production defaults for `https://skillomate.in` and `https://www.skillomate.in`. Existing Authorization and Content-Type headers and GET, POST, PUT, PATCH, DELETE, and OPTIONS methods remain allowed. Wildcard behavior remains limited to development when no origins are configured.

Production resolution examples

```
/api/courses -> https://api.skillomate.in/api/courses
/api/login -> https://api.skillomate.in/api/login
/api/signup -> https://api.skillomate.in/api/signup
api/courses -> https://api.skillomate.in/api/courses
/courses -> https://api.skillomate.in/api/courses
```

4. Files changed by category

Category	Files / scope
Core routing	<code>js/edunex-api.js</code> ; <code>src/lib/apiUrl.js</code> ; <code>src/lib/courseRequest.js</code>
Build and env	<code>vite.config.js</code> ; <code>.env.example</code> ; <code>.env.production</code> ; <code>.gitignore</code> files; <code>_redirects</code>
Frontend consumers	Payment, login, signup, deletion, reports, password recovery, onboarding, video, certificate, and admin modules
Legacy compatibility	<code>payment.html</code> , <code>videos.html</code> , <code>certificates.html</code> , <code>premium-nav.js</code> , and <code>legacy-pages.js</code>
Backend	<code>server.js</code> and backend <code>.env.example</code>
Tests	<code>tests/player/ApiRouting.test.js</code>

5. Verification performed

Check	Result
Frontend suite	PASS - 24 files, 157 tests
Routing test	PASS - normalization, absolute URL preservation, production auth request
Production build	PASS - npm run build; 129 modules transformed
Development-mode build	PASS - empty API base injected, preserving relative proxy behavior
Syntax / diff	PASS - node checks and git diff --check
Built-output scan	PASS - production hostname present in route shells
Forbidden URL scan	PASS - no www.skillomate.in/api and no api.skillomate.in/api/api
Direct fetch scan	PASS - no direct fetch('/api...') bypass in built output

Build note

Vite emitted its existing advisory that the HLS chunk is larger than 500 kB after minification. This is a performance warning, not a build failure, and is unrelated to API routing.

6. Deployment requirement

The production build requires the following public configuration. It is already present in the repository production environment file:

```
VITE_API_BASE_URL=https://api.skillomate.in
```

Deploy the newly generated **edunex-f/dist** contents to the frontend S3 origin and invalidate CloudFront so browsers receive the new HTML shells and JavaScript. Deploy the backend CORS change to all API instances before or with the frontend rollout.

7. Remaining risks

Deployment cache: stale CloudFront HTML or JavaScript can retain the old routing until invalidation completes.

Backend rollout consistency: all three EC2 instances must run the same CORS allowlist code; otherwise requests may fail intermittently behind the ALB.

External configuration: production infrastructure and ALB settings were intentionally not changed. The API DNS, TLS certificate, target health, and deployment environment remain operational prerequisites.

Browser validation: after deployment, verify login, signup/OTP, courses, payment status, admin login, image proxy, and playback from both <https://skillomate.in> and <https://www.skillomate.in> using the browser network panel.

Release conclusion

The new build removes the unintended production dependency on the frontend CloudFront /api path while retaining local proxy behavior and bearer-token authentication. It is ready for coordinated frontend and backend deployment, subject to cache invalidation and post-deployment smoke checks.